

torch.func.functionalize#

<https://pytorch.org/docs/stable/generated/torch.func.functionalize.html>

Description:

functionalize is a transform that can be used to remove (intermediate) mutations and aliasing from a function, while preserving the function's semantics. `functionalize(func)` returns a new function with the same semantics as `func`, but with all intermediate mutations removed. Every inplace operation performed on an intermediate tensor: `intermediate.foo_()` gets replaced by its out-of-place equivalent: `intermediate_updated = intermediate.foo()`. `functionalize` is useful for shipping a pytorch program off to backends or compilers that aren't able to easily represent mutations or aliasing operators. `func` (Callable) – A Python function that takes one or more arguments. `remove` (str) – An optional string argument, that takes on either the value 'mutations' or 'mutations_and_views'. If 'mutations' is passed in then all mutating operators will be replaced with their non-mutating equivalents. If 'mutations_and_views' is passed in, then additionally, all aliasing operators will be replaced with their non-aliasing equivalents. Default: 'mutations'.

Function Signature

```
torch.func.functionalize(func, *, remove='mutations')  
[source]#
```

Parameters

Returns

Return type

There are a few “failure modes” for `functionalize` that are worth calling out:

Returns:

Callable

Code Examples

```
>>> import torch  
>>> from torch.fx.experimental.proxy_tensor import make_fx  
>>> from torch.func import functionalize  
>>>  
>>> # A function that uses mutations and views, but only on  
intermediate tensors.  
>>> def f(a):  
...     b = a + 1  
...     c = b.view(-1)  
...     c.add_(1)
```

```

...     return b
...
>>> inpt = torch.randn(2)
>>>
>>> out1 = f(inpt)
>>> out2 = functionalize(f)(inpt)
>>>
>>> # semantics are the same (outputs are equivalent)
>>> print(torch.allclose(out1, out2))
True
>>>
>>> f_traced = make_fx(f)(inpt)
>>> f_no_mutations_traced = make_fx(functionalize(f))(inpt)
>>> f_no_mutations_and_views_traced = make_fx(functionalize(f,
remove='mutations_and_views'))(inpt)
>>>
>>> print(f_traced.code)

```

```

def forward(self, a_1):
    add = torch.ops.aten.add(a_1, 1); a_1 = None
    view = torch.ops.aten.view(add, [-1])
    add_ = torch.ops.aten.add_(view, 1); view = None
    return add

```

```

>>> print(f_no_mutations_traced.code)

```

```

def forward(self, a_1):
    add = torch.ops.aten.add(a_1, 1); a_1 = None
    view = torch.ops.aten.view(add, [-1]); add = None
    add_1 = torch.ops.aten.add(view, 1); view = None
    view_1 = torch.ops.aten.view(add_1, [2]); add_1 = None
    return view_1

```

```

>>> print(f_no_mutations_and_views_traced.code)

```

```
def forward(self, a_1):
    add = torch.ops.aten.add(a_1, 1); a_1 = None
    view_copy = torch.ops.aten.view_copy(add, [-1]); add = None
    add_1 = torch.ops.aten.add(view_copy, 1); view_copy = None
    view_copy_1 = torch.ops.aten.view_copy(add_1, [2]); add_1 =
None
    return view_copy_1
```

```
>>> # A function that mutates its input tensor
>>> def f(a):
...     b = a.view(-1)
...     b.add_(1)
...     return a
...
>>> f_no_mutations_and_views_traced = make_fx(functionalize(f,
remove='mutations_and_views'))(inpt)
>>> #
>>> # All mutations and views have been removed,
>>> # but there is an extra copy_ in the graph to correctly
apply the mutation to the input
>>> # after the function has completed.
>>> print(f_no_mutations_and_views_traced.code)
```

```
def forward(self, a_1):
    view_copy = torch.ops.aten.view_copy(a_1, [-1])
    add = torch.ops.aten.add(view_copy, 1); view_copy = None
    view_copy_1 = torch.ops.aten.view_copy(add, [2]); add =
None
    copy_ = torch.ops.aten.copy_(a_1, view_copy_1); a_1 = None
    return view_copy_1
```

Detailed Documentation

Documentation

`functionalize` is a transform that can be used to remove (intermediate) mutations and aliasing from a function, while preserving the function's semantics.

`functionalize(func)` returns a new function with the same semantics as `func`, but with all intermediate mutations removed. Every inplace operation performed on an intermediate tensor: `intermediate.foo_()` gets replaced by its out-of-place equivalent: `intermediate_updated = intermediate.foo()`.

`functionalize` is useful for shipping a pytorch program off to backends or compilers that aren't able to easily represent mutations or aliasing operators.

`func` (Callable) – A Python function that takes one or more arguments.

`remove` (str) – An optional string argument, that takes on either the value 'mutations' or 'mutations_and_views'. If 'mutations' is passed in then all mutating operators will be replaced with their non-mutating equivalents. If 'mutations_and_views' is passed in, then additionally, all aliasing operators will be replaced with their non-aliasing equivalents. Default: 'mutations'.

Returns a new "functionalized" function. It takes the same inputs as `func`, and has the same behavior, but any mutations (and optionally aliasing) performed on intermediate tensors in the function will be removed.

`functionalize` will also remove mutations (and views) that were performed on function inputs. However to preserve semantics, `functionalize` will "fix up" the mutations after the transform has finished running, by detecting if any tensor inputs "should have" been

mutated, and copying the new data back to the inputs if necessary.

Like other `torch.func` transforms, `functionalize()` doesn't work with functions that directly use `.backward()`. The same is true for `torch.autograd.grad`. If you want to use autograd, you can compute gradients directly with `functionalize(grad(f))`.

Like other `torch.func` transforms, `functionalize()` doesn't work with global state. If you call `functionalize(f)` on a function that takes views / mutations of non-local state, functionalization will simply no-op and pass the view/mutation calls directly to the backend. One way to work around this is to ensure that any non-local state creation is wrapped into a larger function, which you then call `functionalize` on.

`resize_()` has some limitations: `functionalize` will only work on programs that use `resize_()` as long as the tensor being resized is not a view.

`as_strided()` has some limitations: `functionalize` will not work on `as_strided()` calls that result in tensors with overlapping memory.

Finally, a helpful mental model for understanding functionalization is that most user pytorch programs are writing with the public torch API. When executed, torch operators are generally decomposed into our internal C++ "ATen" API. The logic for functionalization happens entirely at the level of ATen. Functionalization knows how to take every aliasing operator in ATen, and map it to its non-aliasing equivalent (e.g. `tensor.view([-1])` -> `at::view_copy(tensor, [-1])`), and how to take every mutating operator in ATen, and map it to its non-mutating equivalent (e.g. `tensor.add_(1)` -> `at::add(tensor, 1)`), while tracking aliases and mutations out-of-line to know when to fix things up. Information about which ATen operators are aliasing or mutating all comes from `pytorch/pytorch`.

